## Amortized analysis

*Part IV Advanced Design and Analysis Techniques*

### Scope

One operation can be expensive while an entire sequence costs only linear time. Stacks, counters, and dynamic tables develop three amortized methods, with attention to initial credit, every prefix's balance, and payment for copies at growth/shrink boundaries.

### Prerequisites

Operation sequences and total cost.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Chapter route

*Learning objectives*

- Distinguish amortized from average-case cost.
- Apply aggregate, accounting, and potential analysis.
- Prove O(1) amortized cost for doubling tables.
- 17.1 Aggregate analysis
- 17.2 The accounting method
- 17.3 The potential method
- 17.4 Dynamic tables

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Aggregate analysis

Amortized analysis bounds total cost for **every** legal sequence. From a fixed initial state, if every length-m sequence satisfies $\Sigma c_i \leq Am+B$, each operation has $O(1)$ amortized cost. Individual $c_i$ need not be constant, and no random-input averaging is involved.

For an initially empty stack, PUSH costs1, a successful POP costs1, and MULTIPOP(k) removes $q=\min(k,\text{size})$ elements at cost q. Fixed call and empty-test overhead adds $O(1)$ separately.

Each element is pushed once and popped at most once, so total popping work is at most the number of pushes. m operations contain at most m pushes; total element work$\leq 2m$. A single MULTIPOP may cost $\Theta(m)$, yet any complete sequence costs $O(m)$.



## Total binary-counter flips

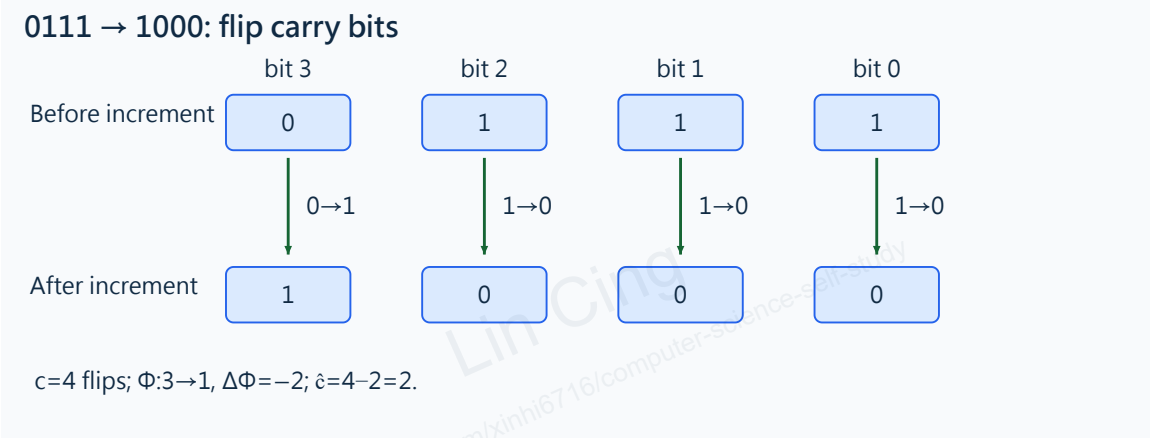
A fixed $b \geq 1$-bit counter starts at all zeros. INCREMENT changes trailing ones to zeros, then the first zero to one. All ones overflows to all zeros. Count bit flips.

Bit j flips once per$2^j$ increments. Over m increments, total flips are

$\Sigma_{j=0}^{b-1} \lfloor m/2^j \rfloor < 2m$.

No independence assumption is needed. Bit0 flips m times, giving $\Omega(m)$, hence total $\Theta(m)$. An implementation scanning all b bits even when unchanged uses a different cost model.

## Total binary-counter flips (continued)




c=4 flips; Φ:3→1, ΔΦ=−2; ĉ=4−2=2.



## A geometric bound for doubling tables

First allow insertion only, starting at(n,M)=(0,0). Allocate one slot initially, then double when full. Cost is one insertion write plus the number of old elements copied; fixed allocation/release overhead is constant.

After m≥2 insertions, the last copy size L is the largest power of two smaller than m. Copy sizes1,2,4,…,L sum to2L−1<2m, so

$T(m)=m+2L-1<3m$.

Total copies are not generally≤m. At m=5, copies1+2+4=7>5 and total cost12. At m=8, copies7 and total15. Only selected endpoints have fewer copies than insertions.

If every new slot must be initialized, count that work too. Its geometric sum remains O(m), but the precise constant3 changes.

## The accounting method

Charge an amortized fee and retain income minus actual spending as credit. Prove that credit is nonnegative for **every prefix** before using total fees to bound actual cost.

For an empty stack, charge PUSH2 and POP/MULTIPOP0. One unit pays insertion and one stays with the element for its eventual removal. A nonexistent element cannot be popped, so no prefix overspends. Add a fixed fee if calls have fixed overhead.

Charge each counter increment2. A0→1 flip spends1 and stores1 on that bit. A1→0 flip uses its stored credit. All-ones overflow is paid entirely from old credit. All-zero initialization needs none; a nonzero initial state requires enough initial credit for its one bits.



## Doubling-table fees and the actual sequence

Charge3 per insertion. The aggregate prefix bound T(m)<3m gives B_m=3m−T(m)≥0. Starting empty:

| Operation | Final(n,M) | Copies | c_i | Total cost | Credit B_i |
|---|---|---|---|---|---|
| 1 | (1,1) | 0 | 1 | 1 | 2 |
| 2 | (2,2) | 1 | 2 | 3 | 3 |
| 3 | (3,4) | 2 | 3 | 6 | 3 |
| 4 | (4,4) | 0 | 1 | 7 | 5 |
| 5 | (5,8) | 4 | 5 | 12 | 3 |
| 6 | (6,8) | 0 | 1 | 13 | 5 |
| 7 | (7,8) | 0 | 1 | 14 | 7 |
| 8 | (8,8) | 0 | 1 | 15 | 9 |

## Doubling-table fees and the actual sequence (continued)

Capacity2→4 occurs at operation3, not4. Copies cannot be shifted to a later operation. The next section's potential inequalities also prove the credit bound step by step.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## The potential method

For state D_i, define potential Φ(D_i) and amortized cost

$\hat{c}_i=c_i+\Phi(D_i)-\Phi(D_{(i-1)})$.

Summing m operations telescopes:

$\Sigma c_i=\Sigma \hat{c}_i+\Phi(D_0)-\Phi(D_m)$.

If every reachable final state has $\Phi(D_m)\geq\Phi(D_0)$, total actual cost≤total amortized cost. A common sufficient condition is $\Phi(D_0)=0$ and $\Phi(D_i)\geq 0$. Positive initial potential instead generally yields $\Sigma c_i\leq\Sigma\hat{c}_i+\Phi(D_0)$; do not discard that initial term.

Adding a constant to all potentials changes no $\hat{c}_i$. The key is the final potential's lower bound relative to the initial one, not absolute nonnegativity everywhere. Individual $\hat{c}_i$ can be negative when stored potential subsidizes the operation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Stack and counter potential proofs

For an empty stack use $\Phi$=size. PUSH has $c=1, \Delta\Phi=1, \hat{c}=2$. Removing q elements has $c=q, \Delta\Phi=-q, \hat{c}=0$. Nonnegative potential and initial0 give element work$\leq$2m for every m-operation sequence.

For a counter use $\Phi$=number of one bits. Flipping t trailing ones and then a zero gives $c=t+1, \Delta\Phi=1-t, \hat{c}=2$. All-ones overflow gives $c=b, \Delta\Phi=-b, \hat{c}=0$. All-zero initialization therefore gives total flips$\leq$2m.

For arbitrary b-bit initialization, $\Phi(D_0)\leq b$, so flips$\leq$2m+b. Starting at all ones makes the first increment cost b, consistent with this bound. Arbitrary initial states do not retain the2m prefix bound without initial credit.



## Dynamic tables

Now support insertion and deletion at the end. Capacities remain powers of two under these exact rules.

```
insert:
    if M==0: allocate M=1
    elif n==M: allocate 2M; copy n items; M=2M
    write new item; n++
delete-last:
    require n>0
    n--
    if n==0: free storage; M=0
    elif M>=4 and n<=M/4:
        allocate M/2; copy n items; M=M/2
```

We shrink at post-deletion $n\leq M/4$, slightly different from the textbook's strict-below-quarter trigger. The exact values below are proved for our rule, not borrowed across boundary conventions. Successful deletion has basic cost1 plus n copied elements if shrinking.

## Shrinking needs a buffer

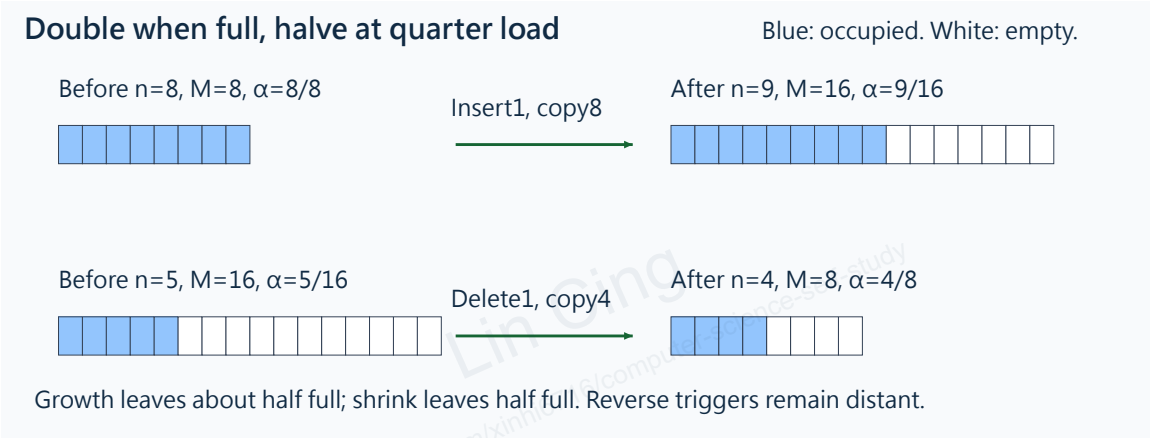
Shrinking as soon as half full causes thrashing. Start full at(k,k), with sufficiently large power-of-two k. Insert once, copying k to reach(k+1,2k). Delete once and immediately shrink back to capacity k, copying k again. Repeating costs Θ(k) per pair and Θ(mk) over m operations.

A quarter-full shrink instead leaves approximately half-full tables after both growth and shrink. Θ(M) intervening deletions are needed to shrink after growth, or insertions to grow after shrink. This allows credit accumulation; the following piecewise potential proves all mixed sequences rather than relying on intuition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## Shrinking needs a buffer (continued)




Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Piecewise potential

Set $\Phi(0,0)=0$. For nonempty tables with load $\alpha=n/M$,

```
Φ(n,M) = 2n−M       if n>=M/2
         M/2−n      if n<M/2
```

Both pieces are nonnegative and meet at zero when half full. A full table has $\Phi=M$, enough for its next M copies. A quarter-full table has $\Phi=M/4$, comparable to its shrink copy count.

Using only$2n-M$ fails for deletion: reachable$(n,M)=(3,8)$ gives$-2$ and cannot use initial0/final-nonnegative reasoning. The piecewise potential gives1 there. Exact before/after resize states still need case analysis.



## Insertion cases

First insertion:$(0,0)\to(1,1)$, $c=1,\Phi 0\to 1,\hat{c}=2$.

Without growth, starting at least half full gives $\Delta\Phi=2$ and$\hat{c}=3$. Starting below half full can reach at most exactly half full; continuity of the low-load branch gives $\Delta\Phi=-1$ and$\hat{c}=0$.

A full-table growth goes from$(M,M)$ to$(M+1,2M)$. Actual cost is $M+1$ and potential $M\to 2$, so

$\hat{c}=(M+1)+(2-M)=3$.

Every insertion has $\hat{c}\leq 3$. The argument includes switching branches at half full rather than assuming both states use the same formula.

## Deletion cases

Without shrink, remaining at least half full gives $\Delta\Phi=-2$ and$\hat{c}=-1$. Dropping one element from exactly half full gives $\Phi 0\to 1$ and$\hat{c}=2$. Starting below half full without shrink gives $\Delta\Phi=1$ and$\hat{c}=2$.

A shrink must start with n_old=M/4+1, then n_new=M/4 and capacity M/2. Old $\Phi=M/4-1$; the new table is half full with $\Phi=0$. Actual cost1+M/4 gives

$\hat{c}=1+M/4-(M/4-1)=2$.

Deleting to empty releases storage, costs1, and drops nonnegative potential to0, so$\hat{c}\le 1$. All operations have$\hat{c}\le 3$. Initial potential0 and final potential$\ge 0$ imply actual cost$\le 3m$ for every legal m-operation sequence, deterministically and without probability assumptions.



## Full mixed trace and model boundaries

Insert to n=5, reaching$(n,M,\Phi)=(5,8,2)$, then delete:

| Operation | Before$(n,M,\Phi)$ | After$(n,M,\Phi)$ | Copies | c | $\hat{c}$ |
|---|---|---|---|---|---|
| delete | (5,8,2) | (4,8,0) | 0 | 1 | −1 |
| delete | (4,8,0) | (3,8,1) | 0 | 1 | 2 |
| delete | (3,8,1) | (2,4,0) | 2 | 3 | 2 |
| delete | (2,4,0) | (1,2,0) | 1 | 2 | 2 |
| delete | (1,2,0) | (0,0,0) | 0 | 1 | 1 |

Five insertions cost12 and five deletions8, total20. Total amortized cost also20 because both endpoint potentials are0. Negative amortized cost does not mean negative actual time.

Only end deletion is covered. Deleting in the middle while preserving contiguous order adds $\Theta(n)$ suffix movement, so $O(1)$ amortized cost does not carry over. Different growth/shrink factors or thresholds require rechecking slopes and every boundary case.

## Sources

*Bibliography*

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter17.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study